Studio 1

Introduction to Processing

Objectives

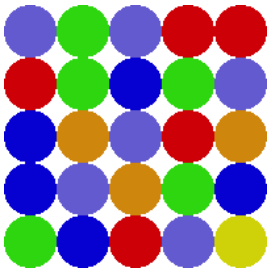
To clarify the course aims and focus. To review basic programming concepts. To learn how to draw graphics in Processing.

Exercise 1

1. Discuss the two course strands.

2. Review the Processing syntax covered in lectures.

3. Draw a simple filled in ellipse in Processing.

4. 

You are helping to write a simple Bejeweled game clone, which has a screen full of coloured jewels that users move to make sets. For this test we will create a grid fill of randomly coloured circles representing jewels, each jewel has a diameter of 50. The sketch will not involve interacting with the jewels at all. The number of columns and rows to draw will be given by constants in the sketch. An array of colours will be provided in the code for the sketch.

Write the pseudo-code to draw the jewels.

5. Write the method header for a method called drawGrid which is passed the number of columns and rows to draw.

6. Write the code for the drawGrid method based on your pseudo-code in Ex 4.

7. Load and display an image in Processing.

Exercise 2

This exercise is the same as the drawing tool you did in COMP103 but you will now write the code in processing.

1. Create a new Processing sketch. Create the setup method and set the size of the room to be 800 by 600. Then create a draw method and draw a line from 50,100 to 400, 200.

2. Change the draw line method call in the draw method so that it draws a line from 50,100 to the x and y position of the mouse. Hint: the built-in variables mouseX and mouseY will give you the x and y position of the mouse.

3. At the moment you have no control over when the line is being drawn. So we can check if the mouse button is pressed and if that is true then draw the line. Add an if statement in the draw method that checks if the built-in mousePressed variable is equal to true. The pseudo-code is:

```
IF mousePressed == true THEN
 Draw the line
ENDIF
```

   Run the sketch and make sure it works correctly.

4. The random method will return back a number between 0 and the number passed to the method - 1. I.e. random(10) will return a random number between 0 and 9, inclusive. Change the code which draws the line so that the start x and y position is not 50, 100 rather a random number between 0 and the width of the sketch window for x and 0 and the height of the sketch window for y. Hint: the built in variables *width* and *height* will give the width and height of the sketch window.

5. Change the draw method so that instead of drawing a line when the mouse button is pressed it draws an ellipse at the x and y position of the mouse with a width and height of 50.

6. Change the draw method so that the width and height of the ellipse being drawn in a random number between 0 and 50.

7. The shapes end up being all sizes and are no longer perfect circles. Before drawing the ellipse we can generate a random size value and use the value for the width and height when drawing the ellipse. The pseudo-code is:

```
Declare size variable as float
IF mousePressed == true THEN
   Set size to random value between 0 and 50
   Draw the ellipse using size for width & height
ENDIF
```

The size variable is declared using the float datatype because the random method returns a float value. The float datatype in processing is like the double datatype in C#.

What happens when you swap the mouseX and mouseY variables when drawing the ellipse? Put them back to the correct order before continuing.

8. Lets change the code so that we get filled in circles with no outline. In the if statement just before generating the random size value type in:

```
noStroke();
fill(0);
```

9. You can also supply the fill method with 3 values representing Red, Green and Blue values from 0-255. Try changing the fill method call to the following and see what it does:

```
fill(255, 0, 0);
```

Now change the fill method call to generate a random number between 0 and 255 for each of the 3 values. What happens when you run the sketch.

10. Now you will make your drawing tool like an airbrush, that is when you press the button a series of circles are drawn rather than just one. The pseudo-code is:

```
Declare size variable as float
Declare num circles variable as int
IF mousePressed == true THEN
 Set no stroke
 Set num circles to random value between 1 and 10
 FOR each circle to draw
       Set random fill colour
       Set size to random value between 2 and 10
       Set x position to mouseX + random number
          between -10 and 10
       Set y position to mouseY + random number
          between -10 and 10
       Draw the ellipse using x and y for the
          position and size for width & height
 ENDFOR
ENDIF
```

11. Now you will extend the code so that you can save a screenshot of your sketch window when a key is pressed. The pseudo-code is below and goes right after the if statement:

```
IF mousePressed == true THEN
 ...
ENDIF
IF keyPressed == true THEN
 Save frame with given filename
ENDIF
```

To save the frame, the code is: saveFrame("myPicture.png");

The file will be saved into the folder that contains your sketch code.

You now have your very own drawing tool that you can customise in any way that you want.

Exercise 3 (Studio Verification Exercise)

Download the cute_cat.jpg file from Moodle and then add it to your sketch. Create a new processing sketch and set the size of the window to match the dimensions of the cat photo.

1. Create a PImage object at class scope level and then load the image data from the file you added into your PImage object. DO NOT display the image in the sketch window.

2. Create the following float variables inside the draw method: distance, x, y, diameter and make them all set to 0. Create a color variable called c and create a constant called NUM_DOTS set to 10.

3. Investigate on the processing website how to find the previous mouse position rather than the current mouse position. Investigate the dist() method that will work out the distance between two points.

4. Investigate the get() method that will get a colour value at a given coordinate.

5. In your draw method write the code for the following pseudo-code:

```
IF mousePressed is true THEN
 Calculate and store the distance between the
 previous and current mouse position.
 FOR each dot to draw
       diameter = random diameter for a circle
             between 2 and the distance travelled
       x = mouseX + random(-distance, distance)
       y = mouseY + random(-distance, distance)
       Get the colour from the image at the x and y
             position and store in c
       Set the fill colour to the colour in C with an
             opacity value of 30
       Draw an ellipse at the x and y position with
             diameter for the width and height
 ENDFOR
ENDIF
```

Now run the sketch and click and paint in the sketch window. What happens when you move the mouse really fast and move it really slow? Keep painting!

Exercise 4 (optional but really fun)

Create a new sketch with just a setup method. You are going to draw a fractal which will draw a series of squares in a repeating pattern. A fractal is made up of a series of levels and the further down the levels you go the more squares you draw.

1. Create a method called **drawLevel** which is passed the x and y position of the centre square, the current number of levels and the length of the side of a square. Write the code to set the stroke colour to black and then draw a square centred around the given x and y values with the correct side length.

   In your setup method set the size of the window to 800 x 800, set the background to white. Then call your drawLevel method passing it the middle of the sketch window, 5 for the number of levels and 100 for the side length.

   Run the sketch and you should see 1 square centred around the middle of the sketch window.

2. In your **drawLevel** method after drawing the rectangle write an if statement to check if the number of levels is greater than 0. If that is true you know want to draw smaller squares centred around each corner of the first square. How do you do this using loops? Very difficult!

   A more elegant way is to use recursion. Recursion happens when a method calls itself passing different values. We only want to call the method again if the levels is greater than 0. If you don't do this it will call it self for infinity and blow up your computer!

   So in your if statement call **drawLevel** again passing it the coordinates of the top left hand corner of the original square, pass it levels – 1 for the number of levels and also pass it half the side length for the next squares so that they are smaller.

   Run the sketch again and you should now see all the square being drawn for the top left hand corner. In the if statement call the method for the remaining corners of the original square. How elegant is that! Run and test the code.

3. Before drawing the rectangle, set the fill color to a random fill colour. See what happens. Try playing with the levels and side length values and play around with colour values etc. Pretty cool right!